



Chapter 6: Database Design Using the E-R Model

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Design Phases

1. Initial Phase: Understand User Needs

Identify and gather the data requirements of users who will use the database.

2. Second Phase: Choose a Data Model

Select the appropriate data model (e.g., relational, document-based).

Apply the concepts from the chosen model to structure the data.

Create a conceptual schema that outlines how the data will be organized and how it meets the users' needs.

Operations and Transactions

Define the types of operations (e.g., adding, updating, deleting data) that users will perform on the data.



Design Phases (Cont.)

Final Phase: Implementation of the Database

Logical Design

Database Schema Decision: Determine the structure of the database.

Business Consideration: Identify which attributes (data fields) are important to record.

Technical Consideration: Define the relation schemas (tables) and decide how to distribute attributes among them.

Physical Design

Layout Decisions: Plan how the data will be stored physically on the hardware. Consider factors like storage efficiency, access speed, and maintenance.

What is a Schema in a Database?

A schema is the structure of a database, defining how data is **organized, stored, and related**. It includes tables, views, indexes, constraints, and relationships.

Logical Schema vs. Physical Schema

Type	Definition	Focuses On
Logical Schema	Defines the structure of the database at a conceptual level. It describes tables, columns, relationships, constraints , etc., without considering how data is stored physically.	Data organization (tables, constraints, relationships)
Physical Schema	Describes how data is actually stored in the database. It includes file storage, indexing, partitions, and access methods .	Storage details (indexes, partitions, memory allocation)



Design Alternatives

When designing a database schema, it's crucial to avoid two major pitfalls:

1. Redundancy

Issue: Poor design can lead to repeated information being stored in multiple places.

Consequence: This redundancy can create data inconsistency, where different copies of the same information may not match, leading to errors and confusion.

2. Incompleteness

Issue: A poor design might fail to capture all necessary aspects of the enterprise.

Consequence: This can result in critical information being overlooked or impossible to model, limiting the database's effectiveness.

Choosing Between Good Designs

Simply avoiding bad designs isn't enough. There are often many good design options available.

It's important to evaluate these options carefully to select the best one that meets the needs of the enterprise while minimizing redundancy and incompleteness



Design Approaches

1. Entity-Relationship Model (ER Model)

Concept: Represents an enterprise as a collection of entities and their relationships.

Entities: Distinguishable "things" or "objects" within the enterprise, defined by a set of attributes (e.g., a customer, a product).

Relationships: Associations between multiple entities, illustrating how they interact.

Visualization: Often depicted using an Entity-Relationship Diagram (ERD), which graphically shows entities and their relationships.

2. Normalization Theory

Purpose: A formal method to identify and eliminate bad designs in a database schema.



Outline of the ER Model



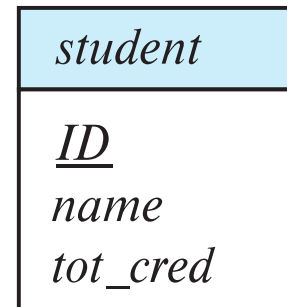
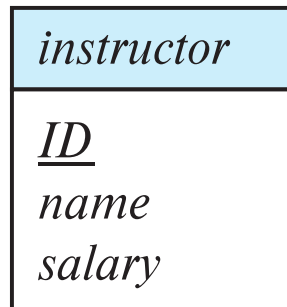
Entity Sets

- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
 - Example:
 $instructor = (ID, name, salary)$
 $course = (course_id, title, credits)$
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.



Representing Entity sets in ER Diagram

- Entity sets can be represented graphically as follows:
 - Rectangles represent entity sets.
 - Attributes listed inside entity rectangle
 - Underline indicates primary key attributes





Relationship Sets

- A **relationship** is an association among several entities

Example:

44553 (Peltier)
student entity

advisor
relationship set

22222 (Einstein)
instructor entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

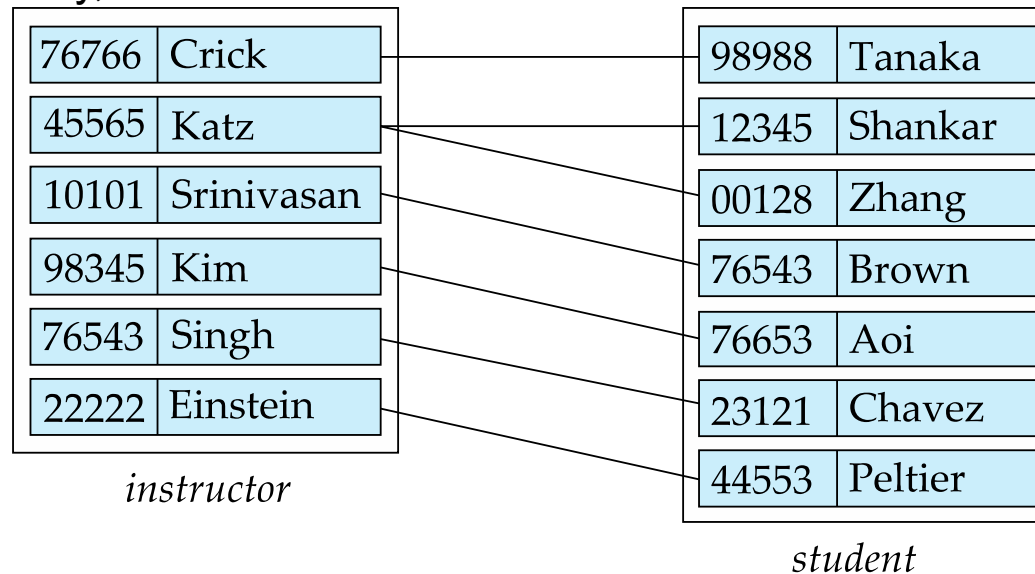
- Example:

$$(44553, 22222) \in \textit{advisor}$$



Relationship Sets (Cont.)

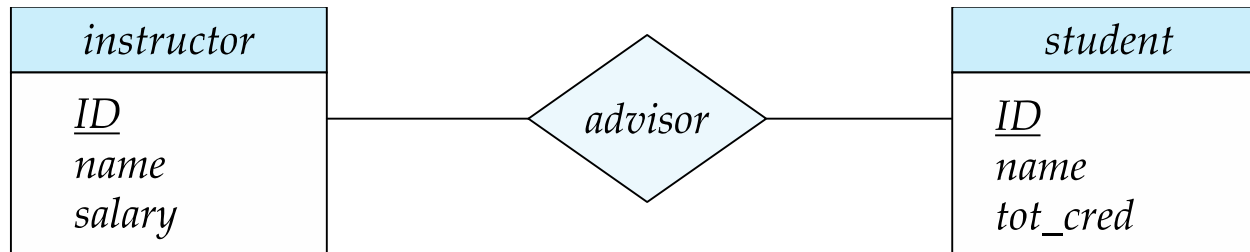
- **Example:** The relationship set Advisor represents the connections between students and the instructors who serve as their advisors. It indicates which instructors are responsible for guiding which students in their academic journey.
- Pictorially, we draw a line between related entities.





Representing Relationship Sets via ER Diagrams

- Diamonds represent relationship sets.

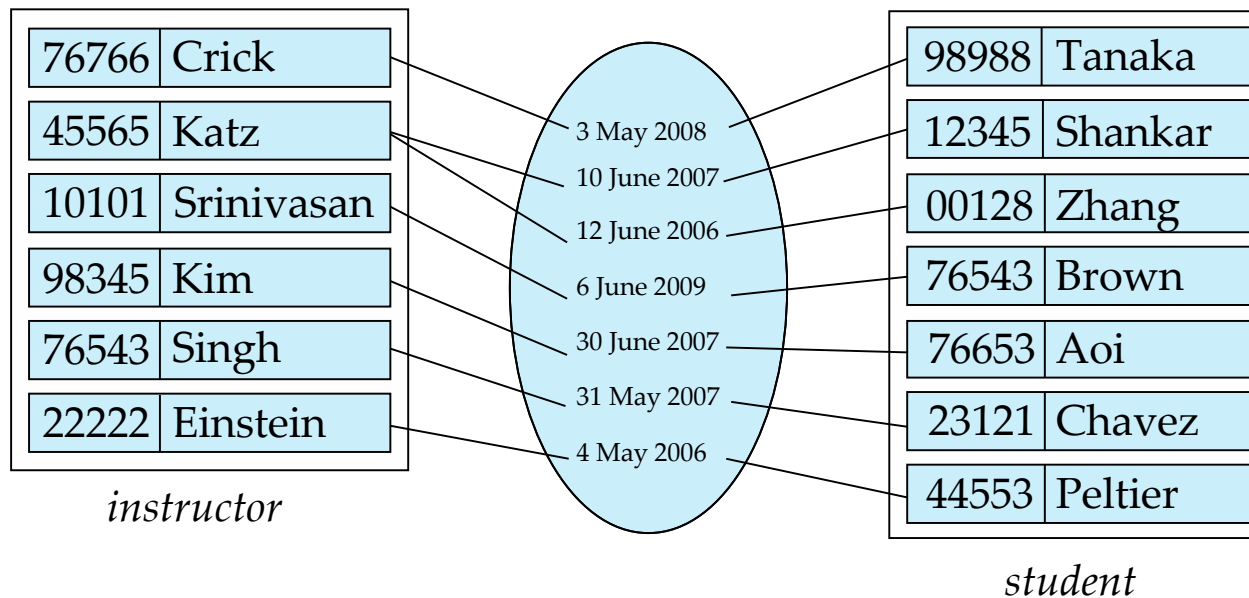




Relationship Sets (Cont.)

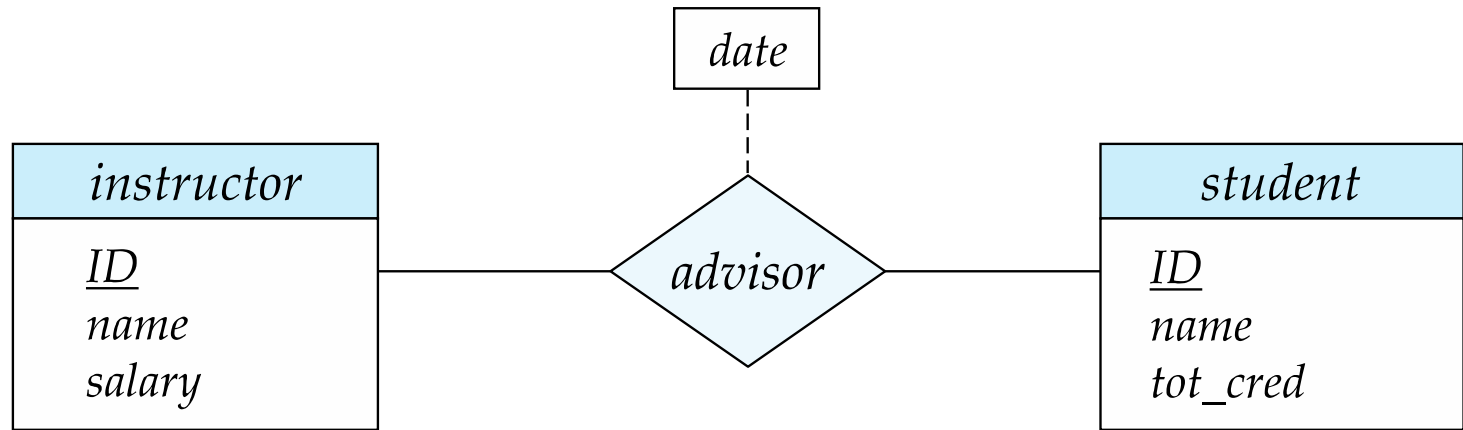
- An attribute can be linked to a relationship set to provide additional information about that relationship.

In the advisor relationship between students and instructors, we can add an attribute called Date. This attribute tracks when a student began their association with their advisor.





Relationship Sets with Attributes

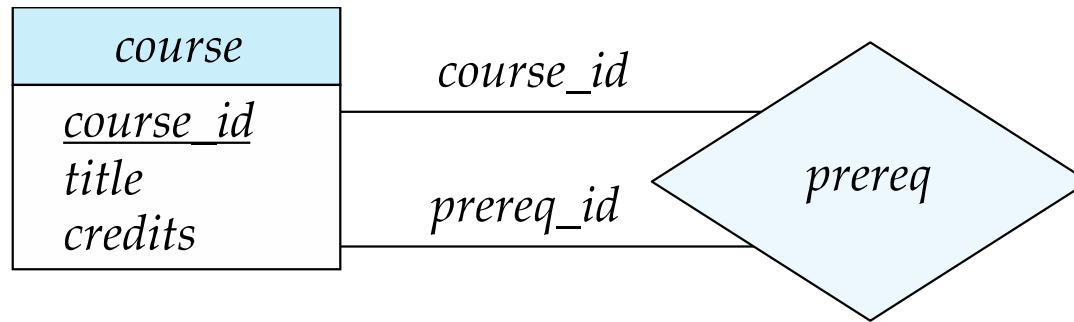




Roles

Entity Sets Can Overlap: In a relationship, the same type of entity can have different roles.

Role Definition: Role defines how an entity participates in a relationship. It describes the function or purpose of an entity in a relationship.



The labels “*course_id*” and “*prereq_id*” are called roles.



Degree of a Relationship Set

Binary Relationships

A binary relationship involves two types of entities (called entity sets).

Common: Most relationships in a database are binary because they usually connect just two entities.

Example of a Binary Relationship:

Students and Instructors: Students can be advised by instructors.

Ternary Relationships

A ternary relationship involves three types of entities.

Less Common: Relationships that connect more than two entities are rare.

Example of a Ternary Relationship:

proj_guide: This relationship connects:

Instructor

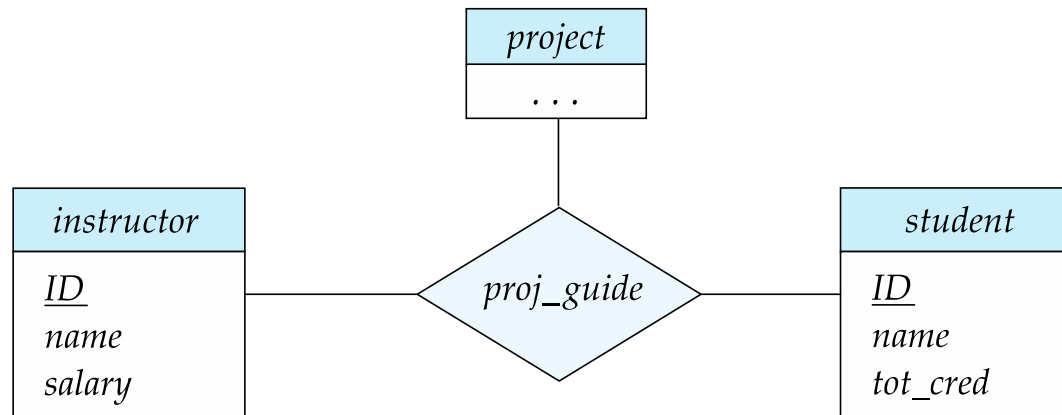
Student

Project



Non-binary Relationship Sets

- Most relationship sets are binary
- There are occasions when it is more convenient to represent relationships as non-binary.
- E-R Diagram with a Ternary Relationship





Types of Attributes

Simple and Composite Attributes

Simple Attributes: Cannot be divided into smaller parts.

Example: first_name, last_name

Composite Attributes: Can be divided into smaller subparts.

Example: full_address (which can include street, city, state, zip_code)

Single-valued and Multivalued Attributes

Single-valued Attributes: Hold only one value for each entity.

Example: email (each person has one email)

Multivalued Attributes: Can hold multiple values for each entity.

Example: phone_numbers (a person can have multiple phone numbers)

Derived Attributes

Derived Attributes: Can be calculated from other attributes.

Example: age (calculated from date_of_birth)

Key Attributes

Key Attributes: Uniquely identify an entity in an entity set.

Example: employee_id, social_security_number

Domain of an Attribute

The domain of an attribute is the set of all possible values it can take. It defines the allowed range or set of values for the attribute.

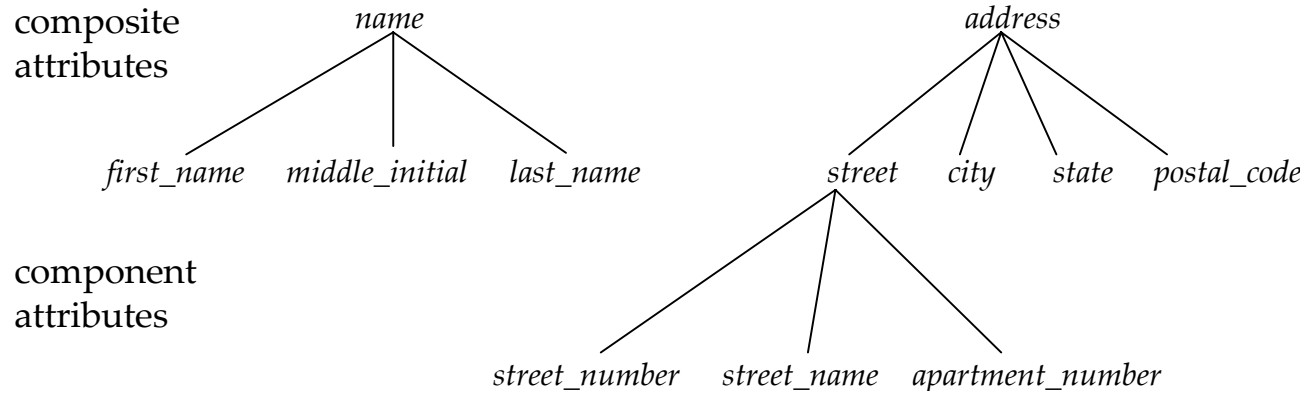
Example: Attribute: Age

Domain: Integer values between 0 and 120.



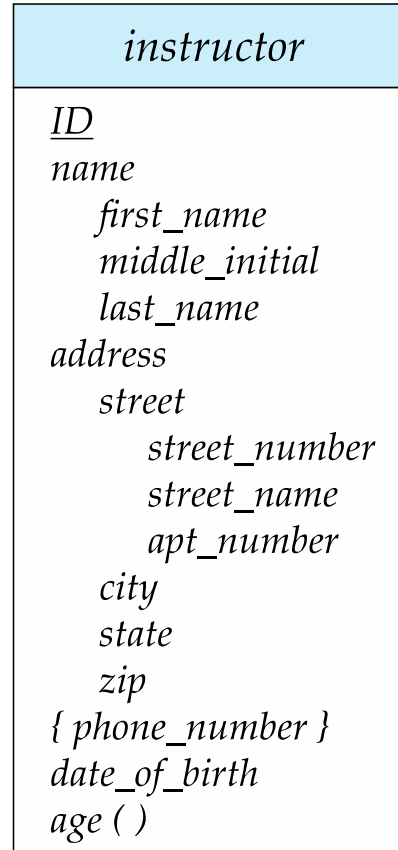
Composite Attributes

- Composite attributes allow us to divided attributes into subparts (other attributes).





Representing Complex Attributes in ER Diagram



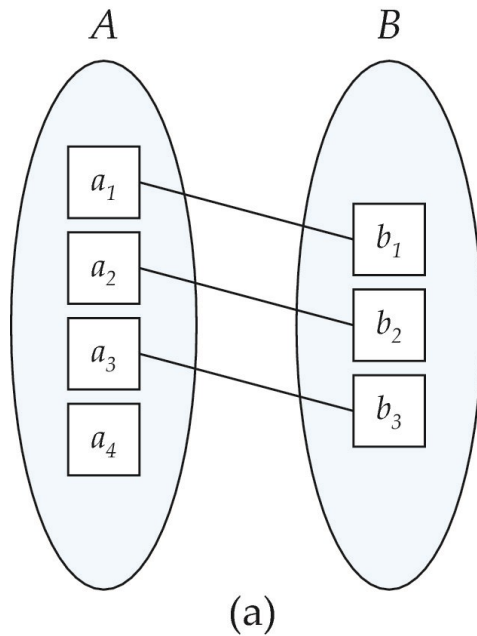


Mapping Cardinality Constraints

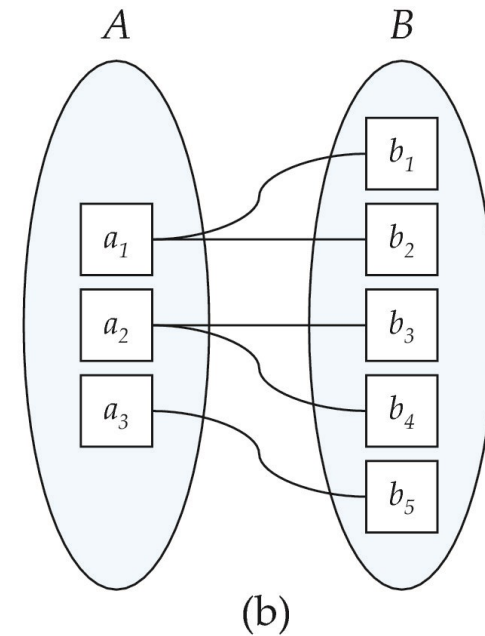
- Mapping cardinality constraints show how many entities from one set can be associated with entities from another set through a relationship.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many



Mapping Cardinalities



One to one

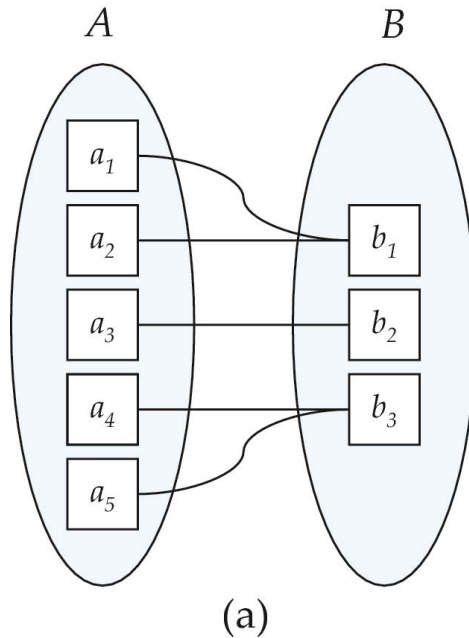


One to many

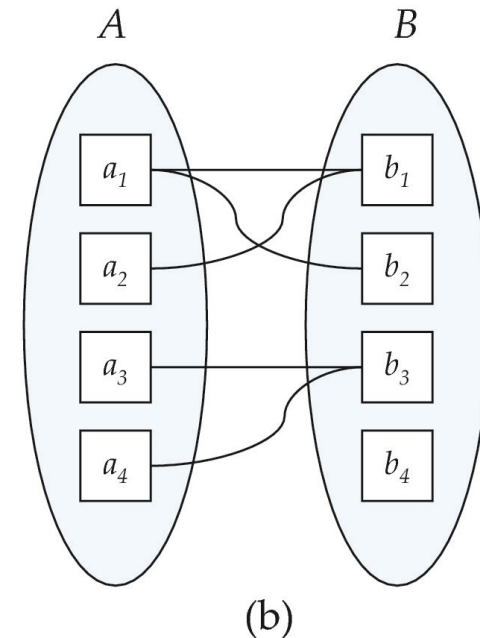
Note: Some elements in A and B may not be mapped to any elements in the other set



Mapping Cardinalities



Many to one



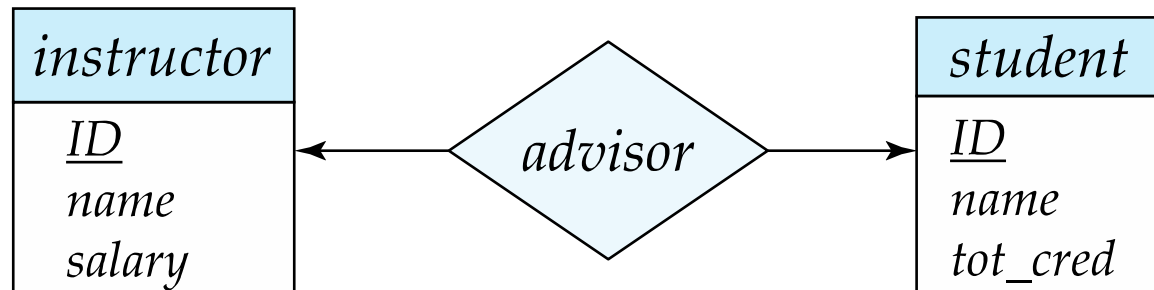
Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set



Representing Cardinality Constraints in ER Diagram

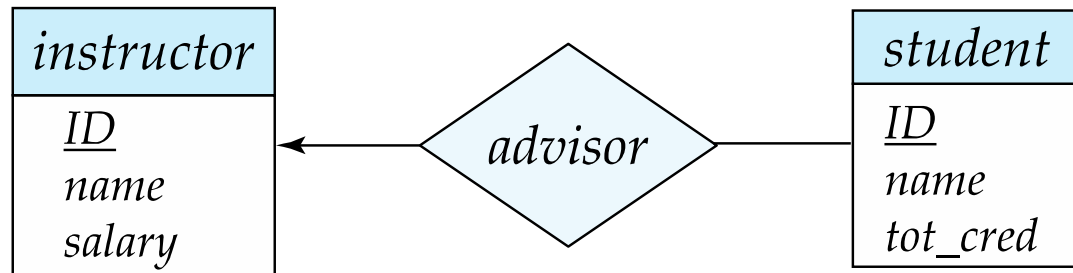
- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($—$), signifying “many,” between the relationship set and the entity set.
- One-to-one relationship between an *instructor* and a *student* :
One entity in set A can be associated with one entities in set B.
 - A student is associated with at most one *instructor* via the relationship *advisor*
 - A *student* is associated with at most one *department* via *stud_dept*





One-to-Many Relationship

- one-to-many relationship between an *instructor* and a *student*
 - an instructor is associated with several (including 0) students via *advisor*
 - a student is associated with at most one instructor via *advisor*,

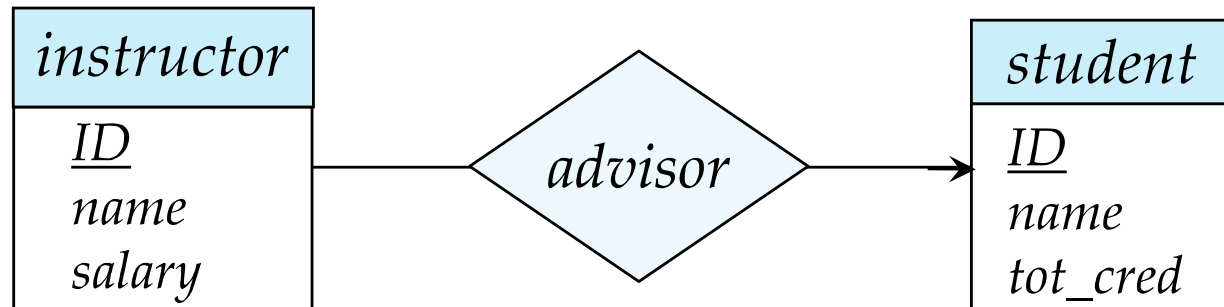


One entity in set A can be associated with many entities in set B, but each entity in set B is linked to only one entity in set A.



Many-to-One Relationships

- In a many-to-one relationship between an *instructor* and a *student*,
 - an instructor is associated with at most one student via *advisor*,
 - and a student is associated with several (including 0) instructors via *advisor*

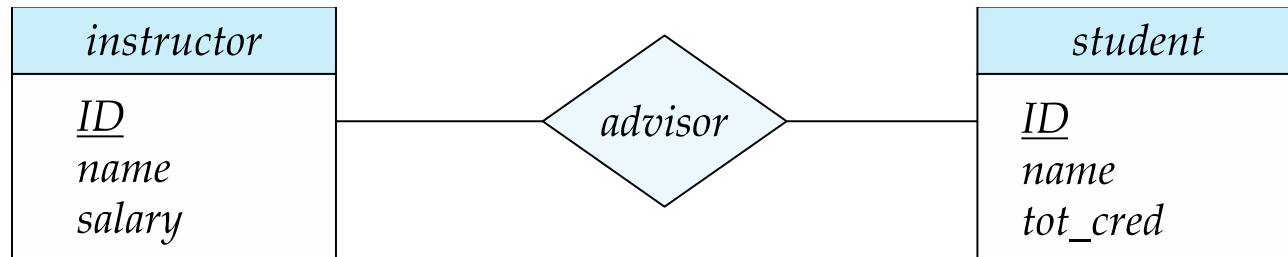


Many entities in set A can be linked to one entity in set B, but that entity in set B can relate to many entities in set A.



Many-to-Many Relationship

- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*

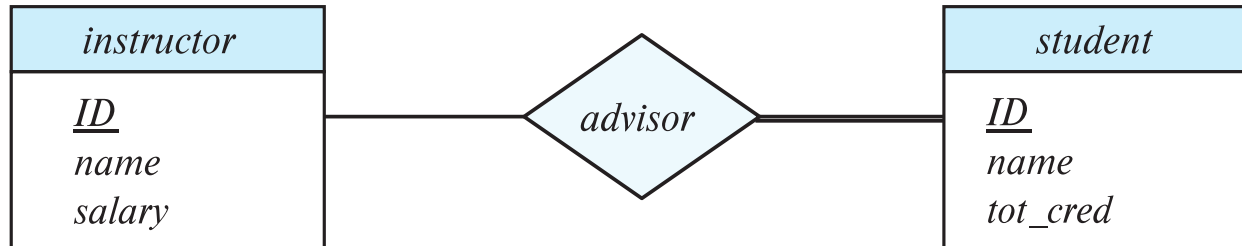


Entities in set A can be associated with many entities in set B, and entities in set B can relate to many entities in set A.



Total and Partial Participation

- **Total participation** (indicated by double line): Every entity in the entity set participates in at least one relationship in the relationship set



Every student must have an associated instructor. Thus, all students participate in this relationship.

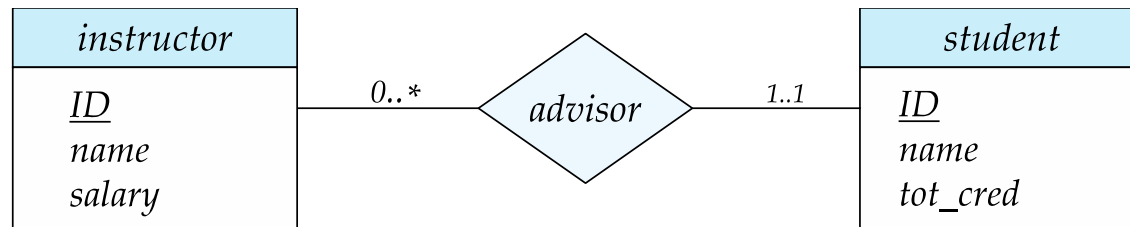
- **Partial participation:** some entities may not participate in any relationship in the relationship set
 - Example: In the Advisor relationship:

Not every instructor has to advise a student.



Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form $l..h$, where l is the minimum and h the maximum cardinality
 - A minimum value of 1 indicates total participation.
 - A maximum value of 1 indicates that the entity participates in at most one relationship
 - A maximum value of * indicates no limit.
- Example



- Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors



Cardinality Constraints on Ternary Relationship

- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- For example, an arrow from *proj_guide* to *instructor* indicates each student has at most one guide for a project
- If there is more than one arrow, there are two ways of defining the meaning.
 - For example, a ternary relationship R between A , B and C with arrows to B and C could mean
 1. Each A entity is associated with a unique entity from B and C or
 2. Each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow



Primary Key

- A primary key is a unique identifier for records in a database table.
Primary keys provide a way to specify how entities and relations are distinguished. We will consider:
 - Entity sets
 - Relationship sets.
 - Weak entity sets



Primary key for Entity Sets

1. Distinct Entities:

Each entity in a database is unique.

2. Expressing Differences:

Differences among entities are represented by their attributes (characteristics).

3. Unique Identification:

The attribute values must be such that they can uniquely identify each entity.

No two entities in an entity set can have the same values for all attributes.

4. Definition of a Key:

A key is a set of attributes that can uniquely distinguish one entity from another within the entity set.

The primary key is a specific type of key chosen to uniquely identify each entity



Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set we use the individual primary keys of the entities in the relationship set.
 - Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$
 - The primary key for R consists of the union of the primary keys of entity sets $E_1, E_2, \dots, E_n$
 - If the relationship set R has attributes $a_1, a_2, \dots, a_m$ associated with it, then the primary key of R also includes the attributes $a_1, a_2, \dots, a_m$
- Example: relationship set “advisor”.
 - The primary key consists of *instructor.ID* and *student.ID*
- The choice of the primary key for a relationship set depends on the mapping cardinality of the relationship set.



Choice of Primary key for Binary Relationship

- Many-to-Many relationships. The preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- One-to-Many relationships. The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- Many-to-one relationships. The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- One-to-one relationships. The primary key of either one of the participating entity sets forms a minimal superkey, and either one can be chosen as the primary key.



Weak Entity Sets

A Weak Entity Set is an entity set that cannot be uniquely identified by its own attributes alone. It depends on a strong entity (also called an owner entity) and has a total participation relationship with it.

Key Features:

No Sufficient Key: A weak entity set does not have a primary key of its own.

Requires a Strong Entity: It is linked to a strong entity set through an identifying relationship.

Discriminator (Partial Key): It has a set of attributes called a discriminator (or partial key) that helps differentiate instances within the same strong entity.

Total Participation: Every weak entity must be associated with a strong entity.



Weak Entity Sets (Cont.)

Example:

Consider a Bank and its Dependent Customers

Strong Entity: Customer (customer_id, name, address)

Weak Entity: Dependent (name, age, relationship)

Discriminator: name (used to differentiate dependents of the same customer)

Identifying Relationship: A Customer owns one or more Dependents.

- A weak entity set is one whose existence is dependent on another entity, called its identifying entity
- Instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called discriminator to uniquely identify a weak entity.



Weak Entity Sets (Cont.)

- An entity set that is not a weak entity set is termed a **strong entity set**.
- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set.
- The identifying entity set is said to **own** the weak entity set that it identifies.
- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.
- Note that the relational schema we eventually create from the entity set *section* does have the attribute *course_id*, for reasons that will become clear later, even though we have dropped the attribute *course_id* from the entity set *section*.



Expressing Weak Entity Sets

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)





Redundant Attributes

A redundant attribute is an attribute that repeats information already stored elsewhere in the database, leading to unnecessary duplication and potential data anomalies.

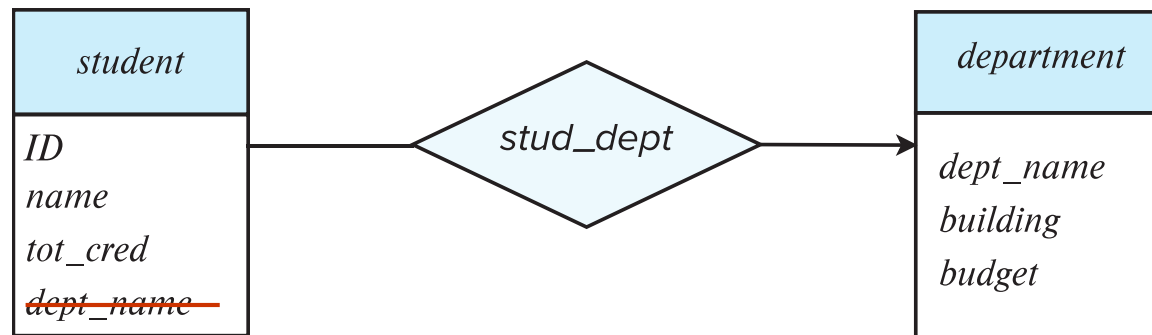
Problems Caused by Redundant Attributes:

Increased Storage Use – Wastes memory by storing the same data multiple times.

Update Anomalies – Changing a value in one place may require multiple updates.

Insertion & Deletion Anomalies – Can cause inconsistencies when adding or deleting data.

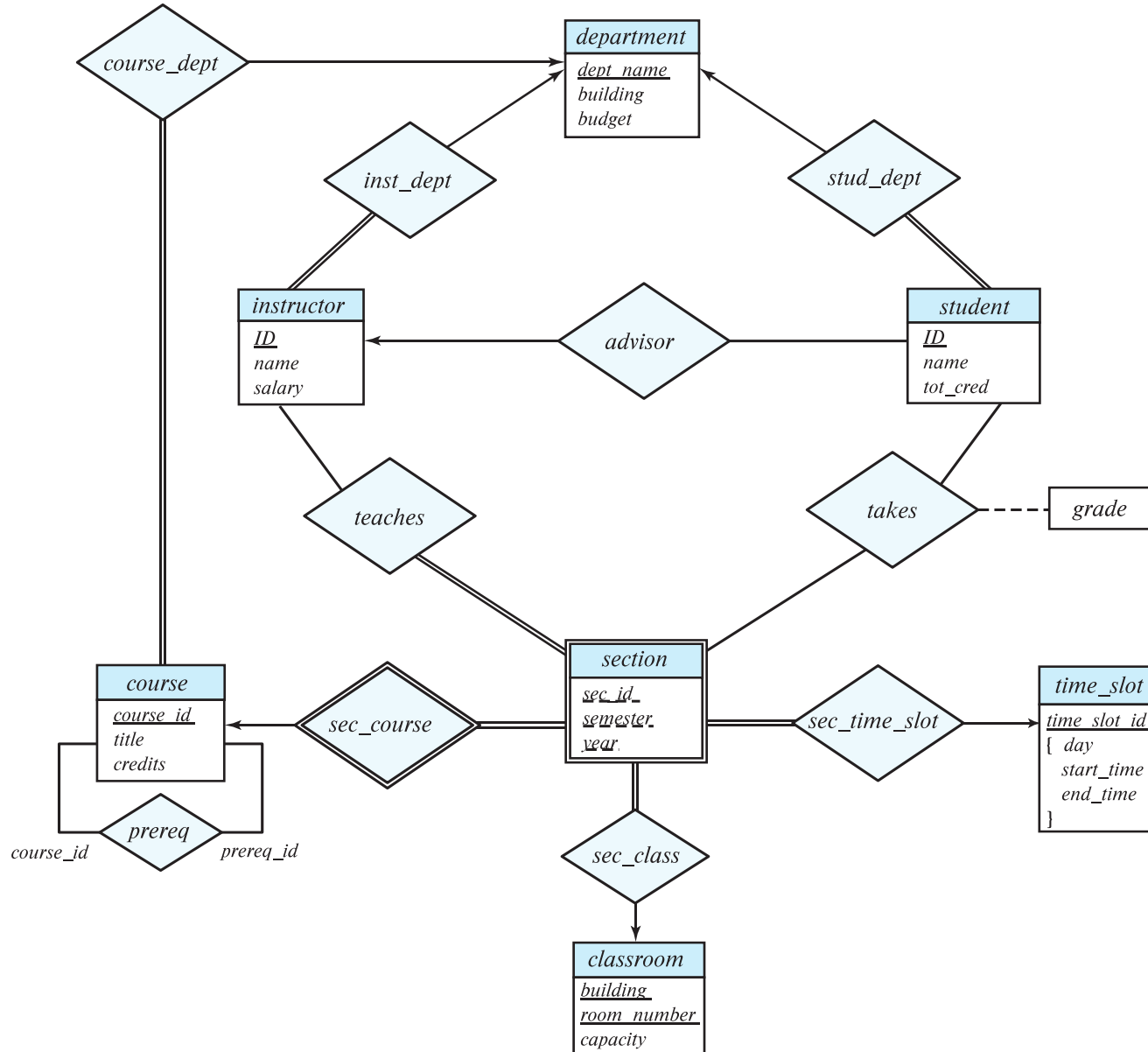
Example:



(a) Incorrect use of attribute



E-R Diagram for a University Enterprise





Reduction to Relation Schemas



Reduction to Relation Schemas

- Entity sets and relationship sets can be expressed uniformly as *relation schemas* that represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of schemas.
- For each entity set and relationship set there is a unique schema that is assigned the name of the corresponding entity set or relationship set.
- Each schema has a number of columns (generally corresponding to attributes), which have unique names.



Representing Entity Sets

- A strong entity set can be represented directly as a table with its own attributes.

student(*ID*, *name*, *tot_cred*)

- A weak entity set cannot be uniquely identified by its own attributes alone. It relies on a strong entity set for identification.

section (*course_id*, *sec_id*, *sem*, *year*)

Here, *course_id* (from the strong entity, e.g., *Course*) is included to help identify each section.

- Example





Representation of Entity Sets with Composite Attributes

<i>instructor</i>
<u>ID</u>
name
<i>first_name</i>
<i>middle_initial</i>
<i>last_name</i>
address
street
<i>street_number</i>
<i>street_name</i>
<i>apt_number</i>
city
state
zip
{ <i>phone_number</i> }
<i>date_of_birth</i>
<i>age</i> ()

- Composite attributes are flattened out by creating a separate attribute for each component attribute
 - Example: given entity set *instructor* with composite attribute *name* with component attributes *first_name* and *last_name* the schema corresponding to the entity set has two attributes *name_first_name* and *name_last_name*
 - Prefix omitted if there is no ambiguity (*name_first_name* could be *first_name*)
- Ignoring multivalued attributes, extended instructor schema is
 - *instructor*(ID, *first_name*, *middle_initial*, *last_name*, *street_number*, *street_name*, *apt_number*, *city*, *state*, *zip_code*, *date_of_birth*)



Representation of Entity Sets with Multivalued Attributes

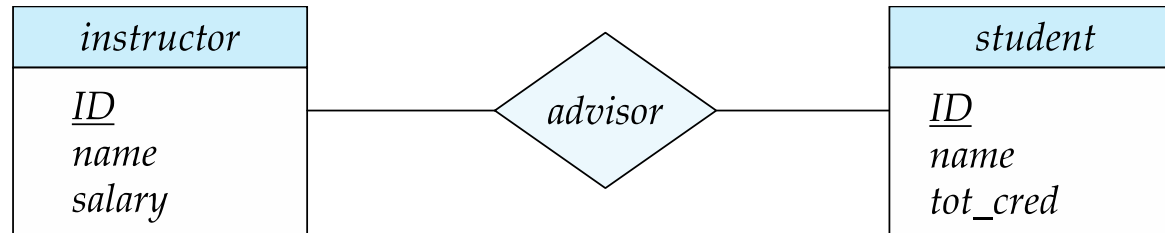
- A multivalued attribute is an attribute that can have multiple values for a single entity. In the relational model, multivalued attributes cannot be stored in the same table as other attributes because it violates 1NF (First Normal Form).
- **Example:** Multivalued attribute *phone_number* of *instructor* is represented by a schema:
$$inst_phone = (\underline{ID}, \underline{phone_number})$$
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema *EM*
 - For example, an *instructor* entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples:
(22222, 456-7890) and (22222, 123-4567)



Representing Relationship Sets

- A many-to-many relationship set is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- Example: schema for relationship set *advisor*

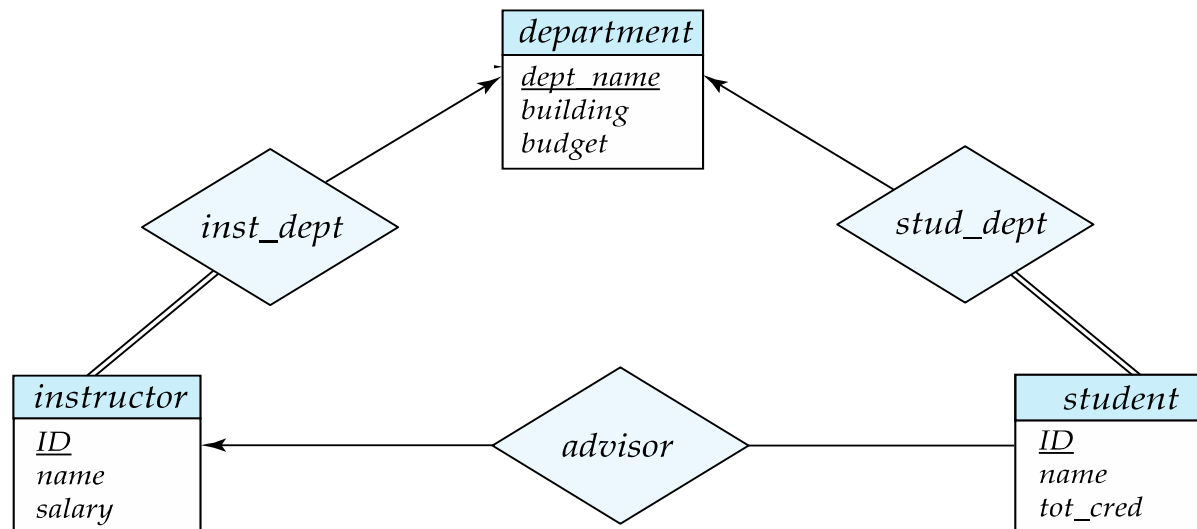
advisor = (s_id, i_id)





Redundancy of Schemas

- Many-to-one and one-to-many relationship sets that are total on the many-side can be represented by adding an extra attribute to the “many” side, containing the primary key of the “one” side
- Example: Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from entity set *instructor*
- Example





Redundancy of Schemas (Cont.)

- For one-to-one relationship sets, either side can be chosen to act as the “many” side
 - That is, an extra attribute can be added to either of the tables corresponding to the two entity sets
- If participation is *partial* on the “many” side, replacing a schema by an extra attribute in the schema corresponding to the “many” side could result in null values



Redundancy of Schemas (Cont.)

- The schema corresponding to a relationship set linking a weak entity set to its identifying strong entity set is redundant.
- Example: The *section* schema already contains the attributes that would appear in the *sec_course* schema





Extended E-R Features



Specialization

Specialization is a top-down design process in database modeling where we break down a general entity into more specific sub-groups based on unique features or roles. Here's how it works:

Create Sub-Groups: Start with a main entity (like Person) and divide it into sub-entities (like Instructor and Student) that have distinct characteristics.

ISA Notation: Use a triangle labeled "ISA" to show that the sub-entities are specific types of the main entity (e.g., "Instructor is a Person").

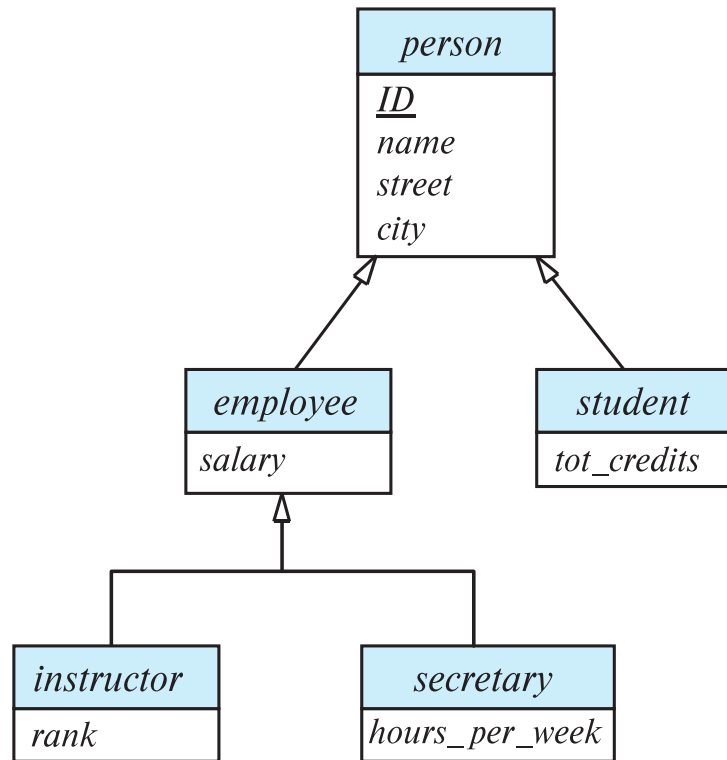
Attribute Inheritance: Each sub-entity inherits all properties of the main entity. This means Instructor and Student inherit common attributes from Person, such as name or address.

Additional Attributes: Sub-entities can have their own unique attributes or participate in relationships that don't apply to the main entity. For example, Instructor might have an attribute salary that does not apply to Student.



Specialization Example

- **Overlapping** – *employee* and *student*
- **Disjoint** – *instructor* and *secretary*
- Total and partial





Representing Specialization via Schemas

- Method 1:
 - Form a schema for the higher-level entity
 - Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- Drawback: getting information about, an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema



Representing Specialization as Schemas (Cont.)

- Method 2:
 - Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

- Drawback: *name*, *street* and *city* may be stored redundantly for people who are both students and employees



Generalization

- **A bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.



Completeness constraint

- **Completeness constraint:** The completeness constraint is a rule that tells us if an entity in the main (higher-level) group must be part of one of the specific sub-groups(lower-level).

Total Completeness: If it's total, every entity in the main group must belong to at least one sub-group. For example, every Person must be either a Student or Employee.

Partial Completeness: If it's partial, an entity in the main group doesn't have to belong to any sub-group. For example, some Persons might not be a Student or Employee.



Completeness constraint (Cont.)

Partial generalization: By default, entities in the main set don't have to belong to any specific sub-group. Partial generalization is the default

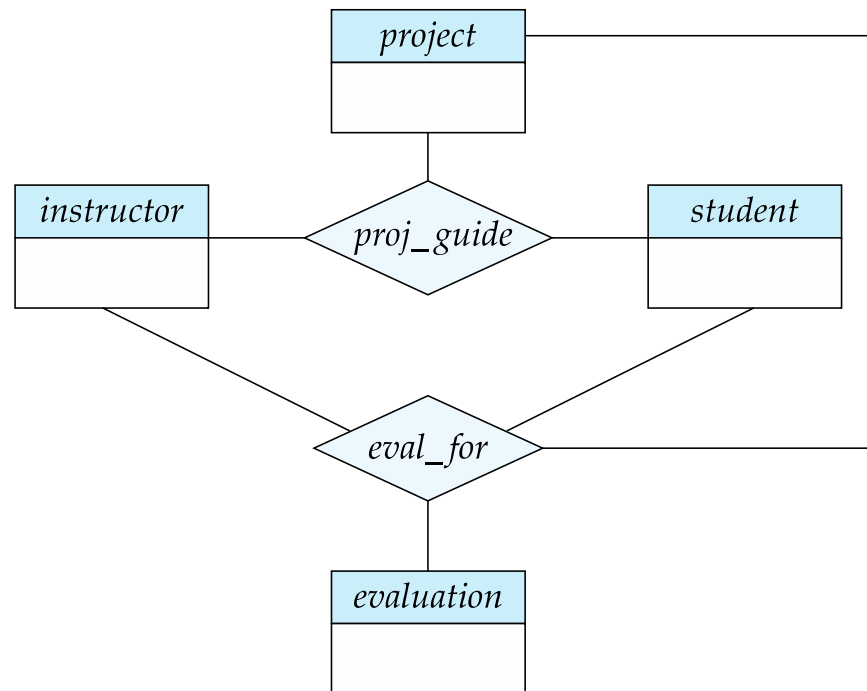
Total generalization: If we want all entities in the main set to belong to a sub-group, we mark this in an ER diagram by writing "total" and connecting it with a dashed line to the arrow(s) pointing to the sub-groups.

Example: In a "student" entity set, we might have two sub-groups: "graduate" and "undergraduate." If the student generalization is total, every student must belong to one of these sub-groups.



Aggregation

- Aggregation in databases is a concept used in Entity-Relationship (ER) modeling where a relationship itself acts as an entity and is linked with another entity. It helps represent higher-level abstractions in complex relationships
- Suppose we want to record evaluations of a student by a guide on a project





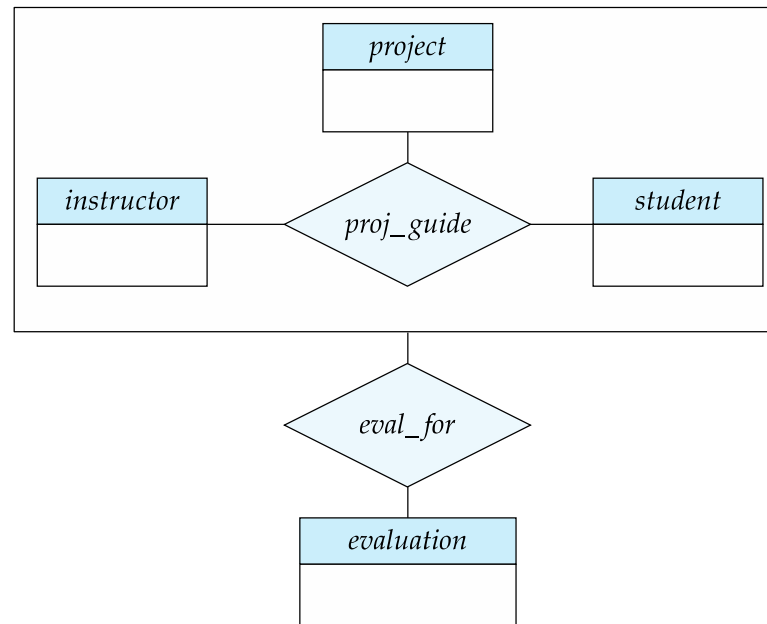
Aggregation (Cont.)

- Aggregation in databases refers to the process of summarizing or grouping data to extract meaningful insights. Relationship sets *eval_for* and *proj_guide* represent overlapping information
 - Every *eval_for* relationship corresponds to a *proj_guide* relationship
 - However, some *proj_guide* relationships may not correspond to any *eval_for* relationships
 - So we can't discard the *proj_guide* relationship
- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity



Aggregation (Cont.)

- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
 - A student is guided by a particular instructor on a particular project
 - A student, instructor, project combination may have an associated evaluation





Reduction to Relational Schemas

- To represent aggregation, create a schema containing
 - Primary key of the aggregated relationship,
 - The primary key of the associated entity set
 - Any descriptive attributes
- In our example:
 - The schema *eval_for* is:
$$eval_for(s_ID, project_id, i_ID, evaluation_id)$$
 - The schema *proj_guide* is redundant.

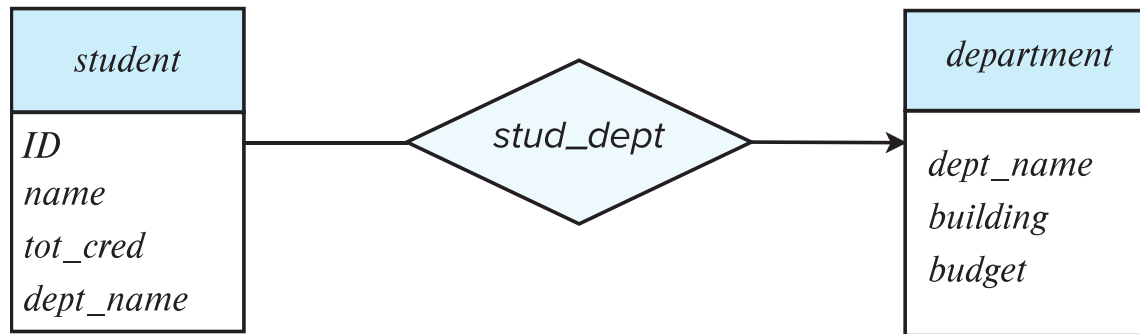


Design Issues

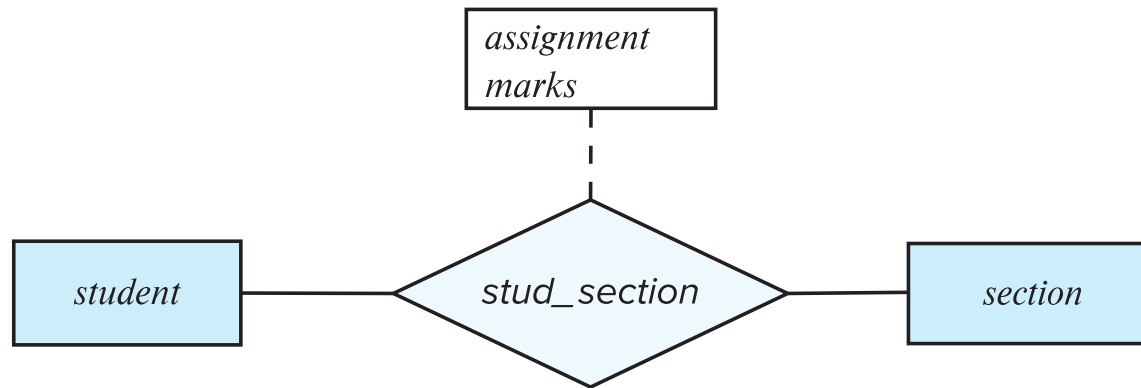


Common Mistakes in E-R Diagrams

- Example of erroneous E-R diagrams



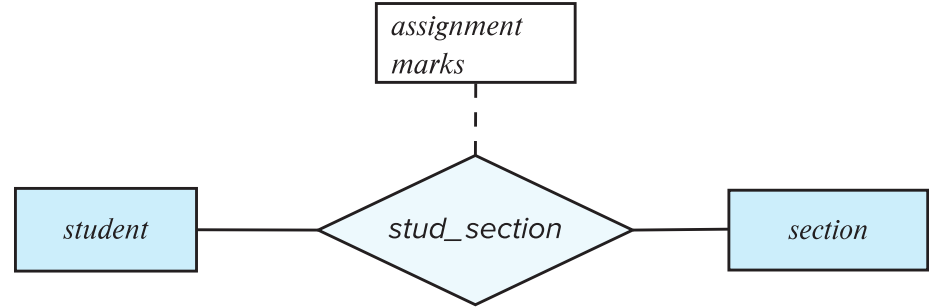
(a) Incorrect use of attribute



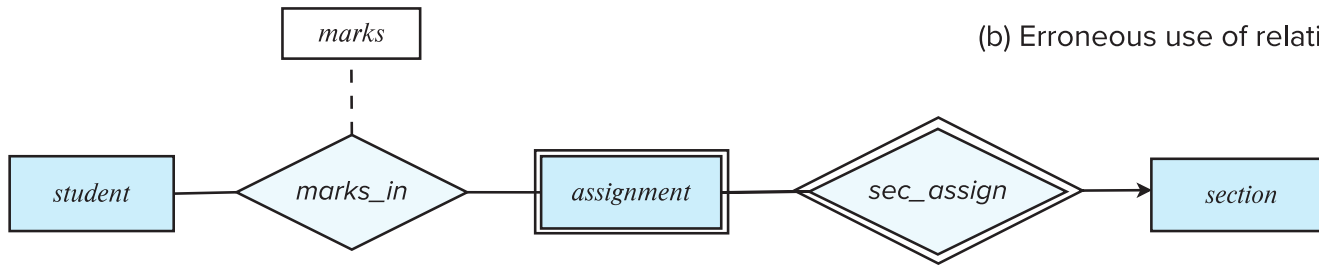
(b) Erroneous use of relationship attributes



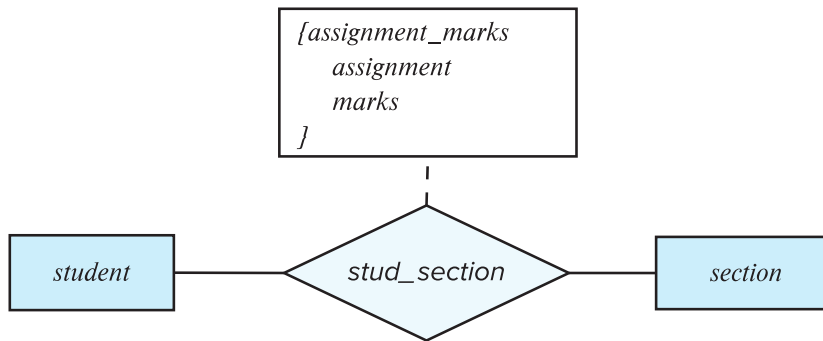
Common Mistakes in E-R Diagrams (Cont.)



(b) Erroneous use of relationship attributes



(c) Correct alternative to erroneous E-R diagram (b)

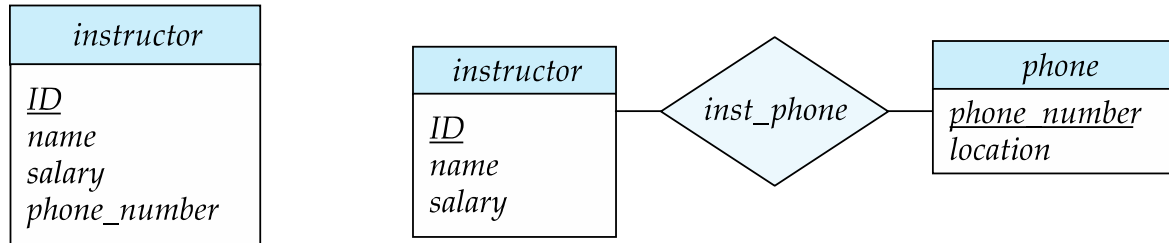


(d) Correct alternative to erroneous E-R diagram (b)



Entities vs. Attributes

- Use of entity sets vs. attributes



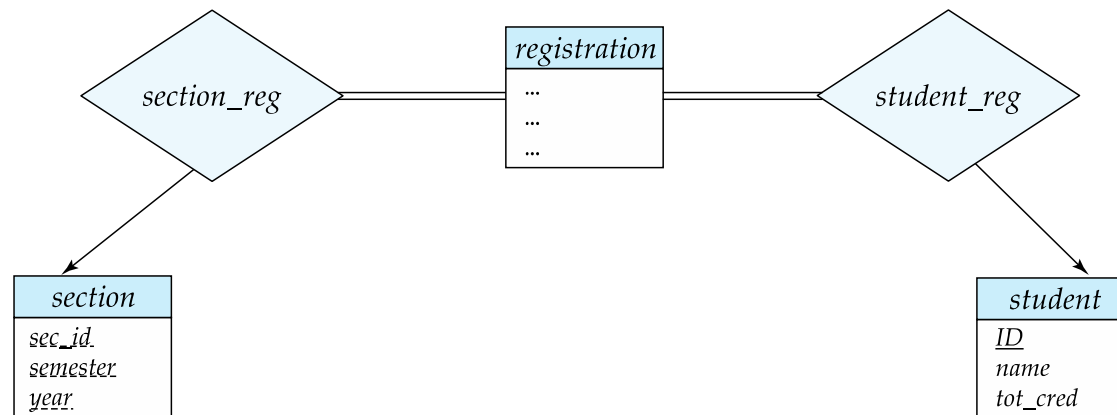
- Use of phone as an entity allows extra information about phone numbers (plus multiple phone numbers)



Entities vs. Relationship sets

- **Use of entity sets vs. relationship sets**

Possible guideline is to designate a relationship set to describe an action that occurs between entities



- **Placement of relationship attributes**

For example, attribute date as attribute of advisor or as attribute of student



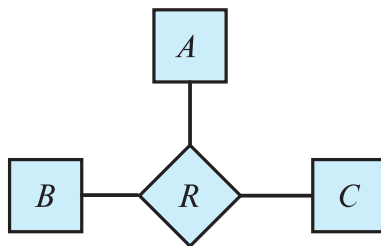
Binary Vs. Non-Binary Relationships

- Although it is possible to replace any non-binary (n -ary, for $n > 2$) relationship set by a number of distinct binary relationship sets, a n -ary relationship set shows more clearly that several entities participate in a single relationship.
- Some relationships that appear to be non-binary may be better represented using binary relationships
 - For example, a ternary relationship *parents*, relating a child to his/her father and mother, is best replaced by two binary relationships, *father* and *mother*
 - Using two binary relationships allows partial information (e.g., only mother being known)
 - But there are some relationships that are naturally non-binary
 - Example: *proj_guide*

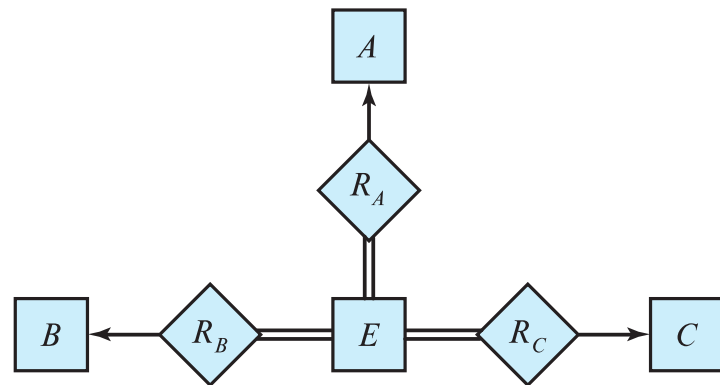


Converting Non-Binary Relationships to Binary Form

- In general, any non-binary relationship can be represented using binary relationships by creating an artificial entity set.
 - Replace R between entity sets A , B and C by an entity set E , and three relationship sets:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C
 - Create an identifying attribute for E and add any attributes of R to E
 - For each relationship (a_i, b_i, c_i) in R , create
 1. a new entity e_i in the entity set E
 2. add (e_i, a_i) to R_A
 3. add (e_i, b_i) to R_B
 4. add (e_i, c_i) to R_C



(a)



(b)



Converting Non-Binary Relationships (Cont.)

- Also need to translate constraints
 - Translating all constraints may not be possible
 - There may be instances in the translated schema that cannot correspond to any instance of R
 - Exercise: *add constraints to the relationships R_A , R_B and R_C to ensure that a newly created entity corresponds to exactly one entity in each of entity sets A , B and C*
 - We can avoid creating an identifying attribute by making E a weak entity set (described shortly) identified by the three relationship sets

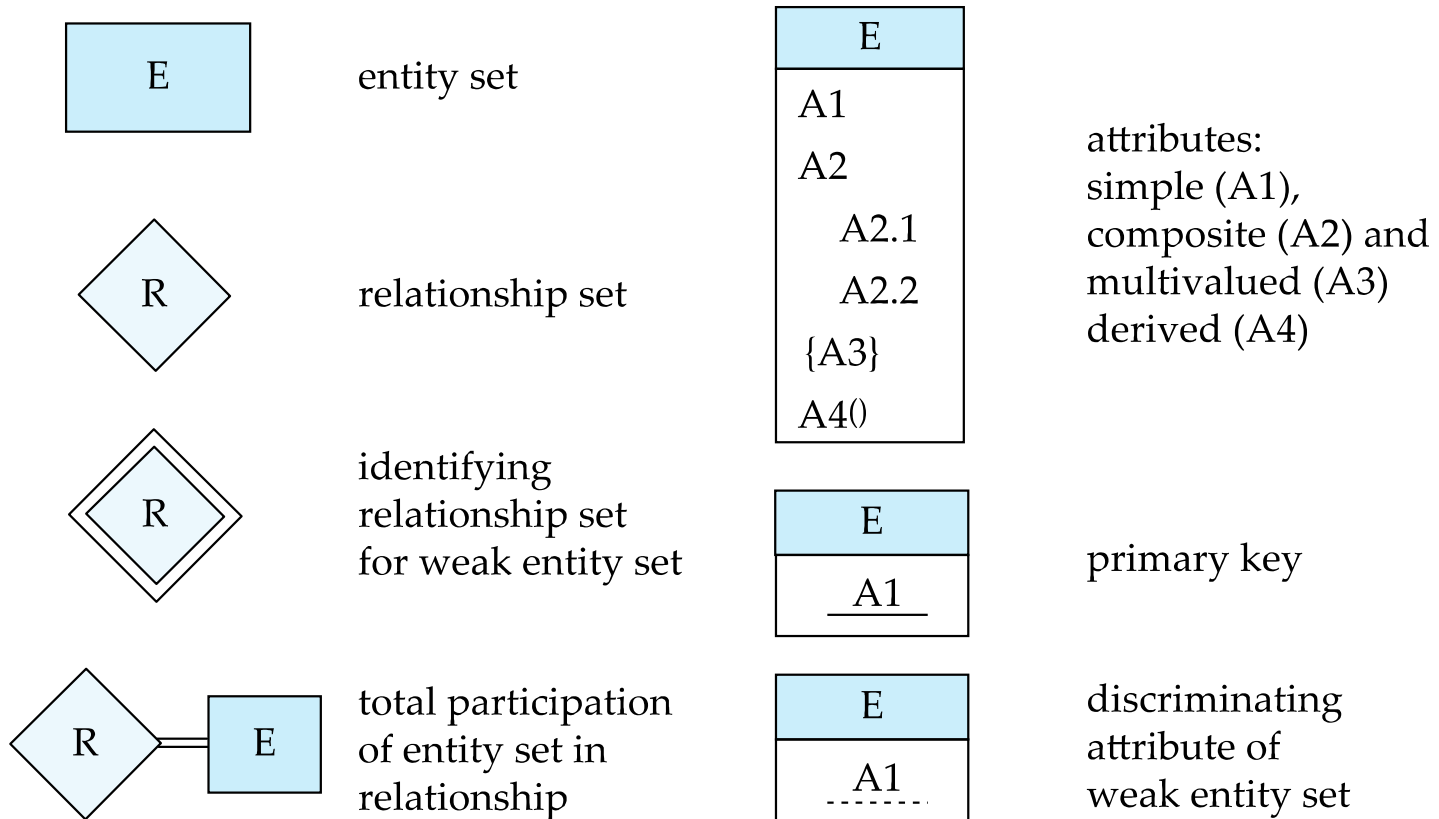


E-R Design Decisions

- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.

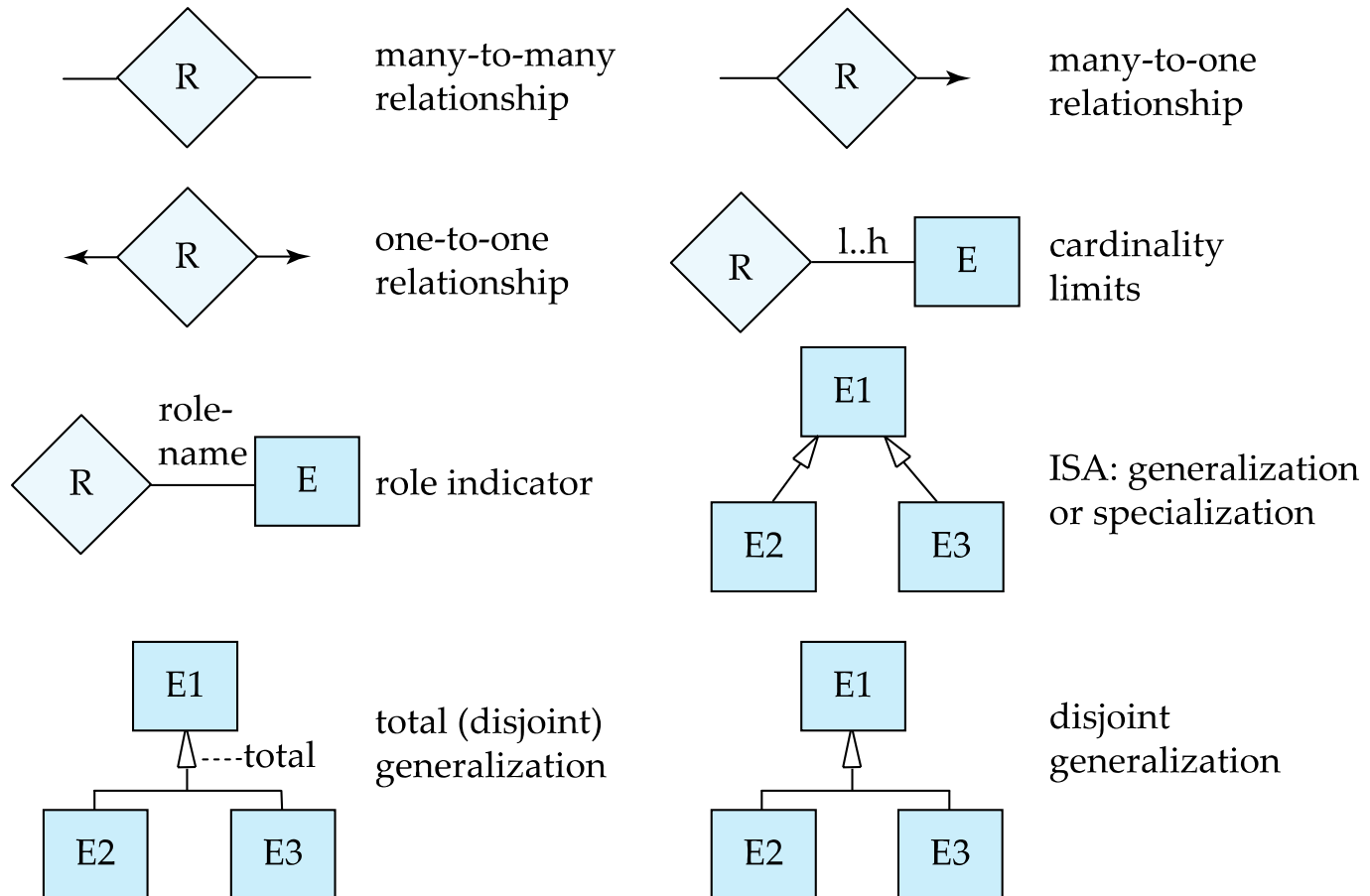


Summary of Symbols Used in E-R Notation





Symbols Used in E-R Notation (Cont.)

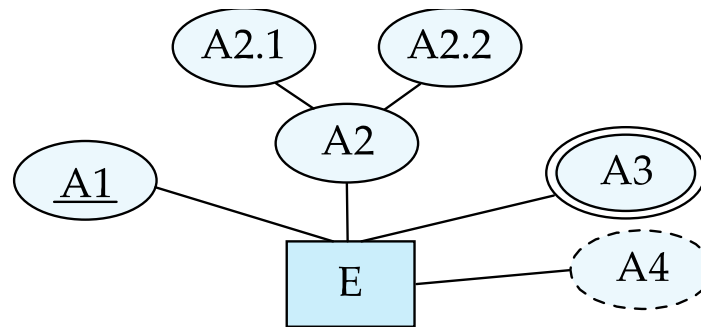




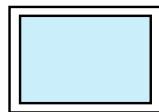
Alternative ER Notations

- Chen, IDE1FX, ...

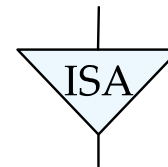
entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



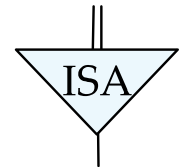
weak entity set



generalization



total
generalization



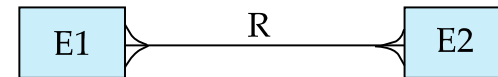
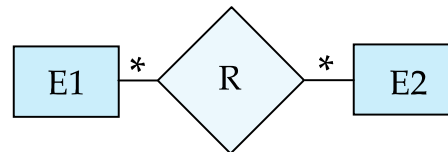


Alternative ER Notations

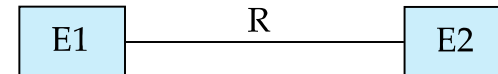
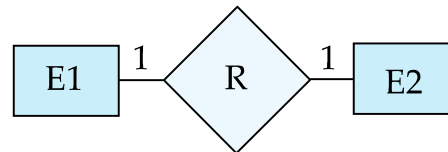
Chen

IDE1FX (Crows foot notation)

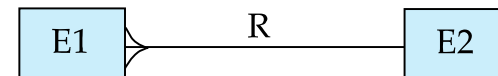
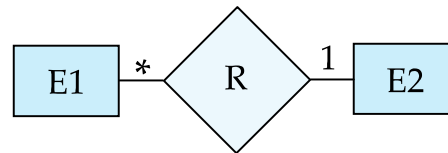
many-to-many
relationship



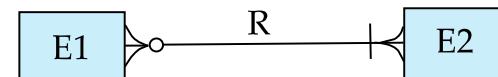
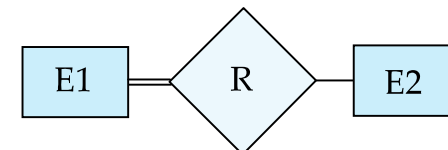
one-to-one
relationship



many-to-one
relationship



participation
in R: total (E1)
and partial (E2)





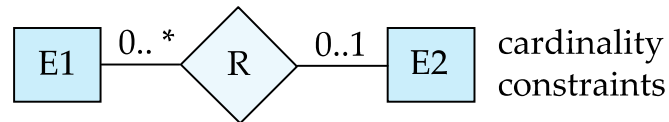
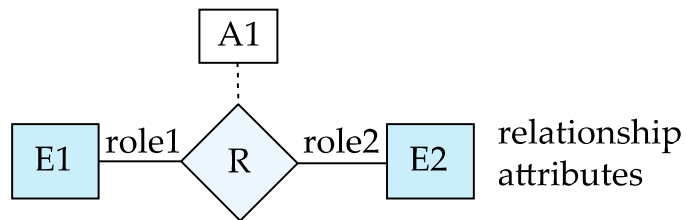
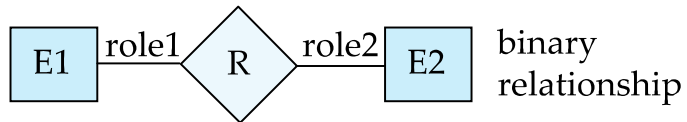
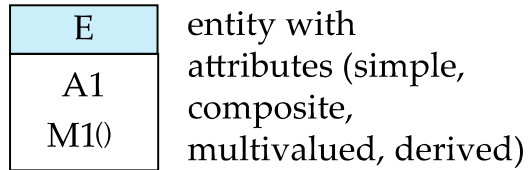
UML

- **UML:** Unified Modeling Language
- UML has many components to graphically model different aspects of an entire software system
- UML Class Diagrams correspond to E-R Diagram, but several differences.

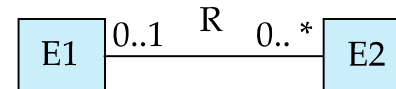
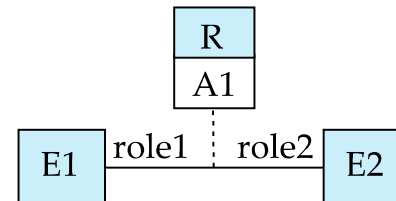
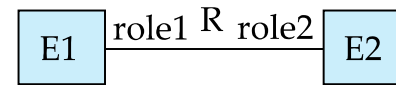
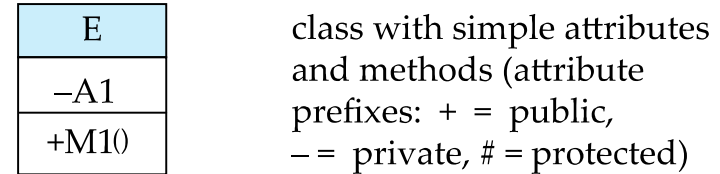


ER vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML

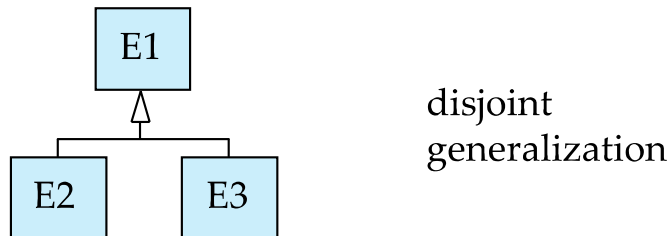
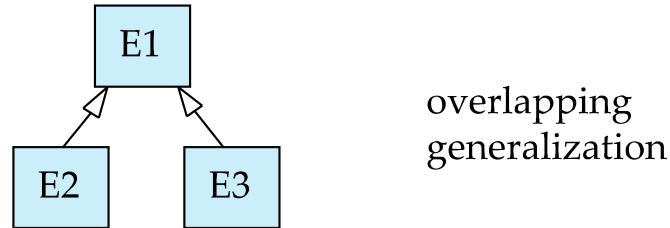
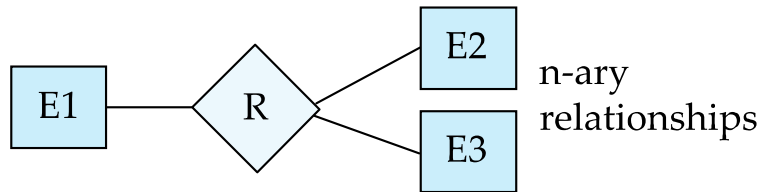


* Note reversal of position in cardinality constraint depiction

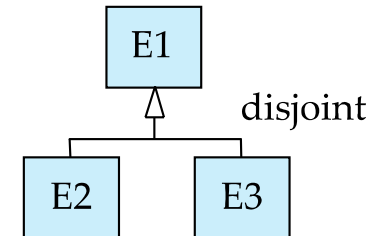
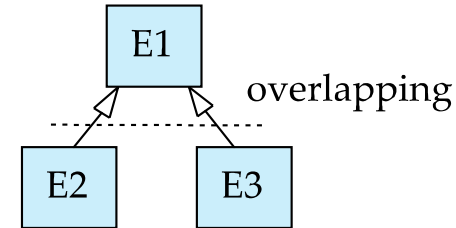
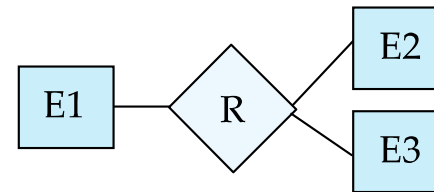


ER vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML



* Generalization can use merged or separate arrows independent of disjoint/overlapping



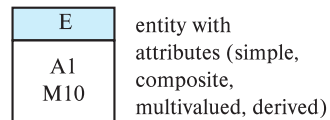
UML Class Diagrams (Cont.)

- Binary relationship sets are represented in UML by just drawing a line connecting the entity sets. The relationship set name is written adjacent to the line.
- The role played by an entity set in a relationship set may also be specified by writing the role name on the line, adjacent to the entity set.
- The relationship set name may alternatively be written in a box, along with attributes of the relationship set, and the box is connected, using a dotted line, to the line depicting the relationship set.

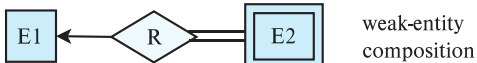
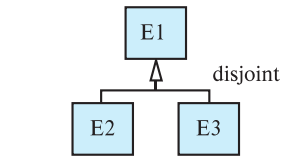
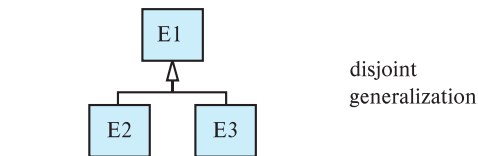
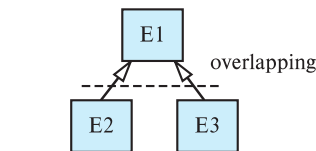
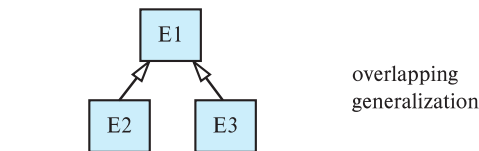
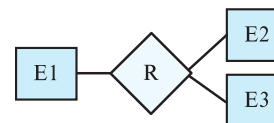
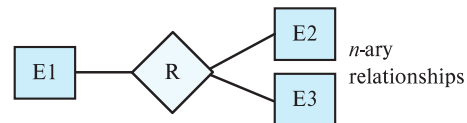
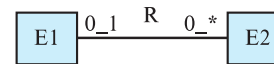
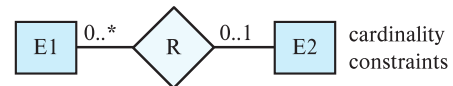
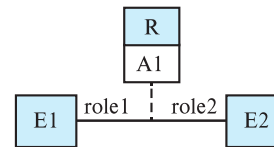
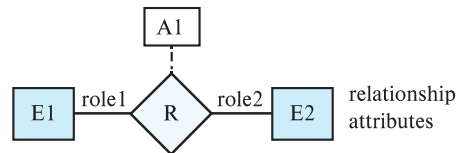
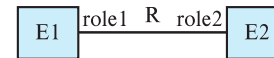
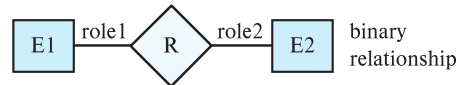
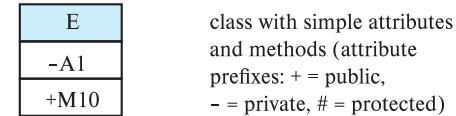


ER vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML





Other Aspects of Database Design

- Functional Requirements
- Data Flow, Workflow
- Schema Evolution



End of Chapter 6